

scripts/autonomous-qa/vision-analyze.sh — User Guide

Last verified: 2026-07-26 (LVA-018 script-docs backfill cycle)
Inheritance: HelixConstitution §11.4.18 (script docs); Lava §6.AB (anti-bluff test-suite reinforcement), §6.J / §6.L (anti-bluff)
Classification: project-specific

Overview

Vision + log analysis of recorded autonomous-QA materials. Reads the RAW media/logs an iteration produced (see run-iteration.sh) and emits a CURATED vision-analysis.md flagging the §6.AB **non-crashing defect classes** that a green JUnit verdict can hide from a real user:

- **blank / white / monochrome render** — color heuristics (ffmpeg signalstats)
- **stuck / frozen screen** — ffmpeg freezedetect
- **wrong screen reached / missing download affordance** — OCR (ONLY if tesseract is present)
- **crashes / ANRs** — logcat.txt grep

Expected raw layout per iteration:

<iter>/raw/{rec_*.mp4 qa_rec_*.mp4}	chunked screenrecord
<iter>/raw/logcat.txt	threadtime logcat
<iter>/raw/gradle-connected.log stdout (secondary cross-ref)	connectedDebugAndroidTest
<iter>/verdict.json (optional cross-ref)	curated JUnit verdict

Anti-bluff contract (§6.J / §6.AB / §6.L)

The script **NEVER prints a defect-free (CLEAN) verdict for analysis it did not actually perform:**

Situation	Result
ffmpeg missing	exit 2 (TOOL-MISSING), no verdict
no analyzable recordings AND no log	exit 2 (CANNOT-ANALYZE)

Situation	Result
recordings missing/ unreadable, or logcat missing	verdict INCOMPLETE , exit 2 (never CLEAN)
tesseract absent	OCR checks SKIPPED + said so plainly; screen-content claims are NOT fabricated

Usage

```
# One iteration -> <dir>/vision-analysis.md
scripts/autonomous-qa/vision-analyze.sh --iteration-dir <dir>
[opts]

# Roll up every iteration subdir -> <backend-dir>/vision-
analysis.md
scripts/autonomous-qa/vision-analyze.sh --cycle-dir <backend-dir>
[opts]
```

Option	Default	Meaning
--keep-frames	delete on exit	keep extracted PNG frames (under raw/.vision-frames/, already gitignored; frames are NEVER committed)
--package <pkg>	digital.vasic.lava.client.dev	app package for the logcat crash grep
--expect-tokens <csv>	download,magnet,torrent,seed,size	OCR affordance tokens to confirm result/download UI
--ocr-frames <N>	24	max frames to OCR per recording
-h --help	—	show help

Detection heuristics (tunable constants at the top of the script)

Constant	Default	Meaning
SAMPLE_FPS	1	frames extracted per second of recording
UNIFORM_FULL_RANGE_MAX	40	$Y_{MAX}-Y_{MIN} \leq \text{this} \Rightarrow$ frame draws $\sim$ nothing (uniform)
WHITE_YAVG_MIN / BLACK_YAVG_MAX	200 / 32	uniform-frame white/black classification
BLANK_RUN_FRAMES	5	$\geq$ this many CONSECUTIVE uniform frames $\Rightarrow$ stuck-blank defect
BLANK_FRACTION_PCT	50	$\geq$ this % of frames uniform $\Rightarrow$ blank-dominated defect
FREEZE_SECONDS / FREEZE_NOISE	10 / -60dB	freezedetect minimum frozen span / noise tolerance

Exit codes

- 0 — **CLEAN** (recordings analyzed, no blank/frozen/OCR/crash defects)
- 1 — **DEFECTS-FOUND**
- 2 — **CANNOT-ANALYZE / INCOMPLETE / TOOL-MISSING**

Cycle-rollup mode exits 1 if any iteration found defects, else 2 if any iteration was INCOMPLETE, else 0; it writes a per-iteration table plus cycle totals (CLEAN=n DEFECTS-FOUND=n INCOMPLETE=n).

Dependencies

ffmpeg (REQUIRED — FFMPEG env override), tesseract (optional — OCR), imagemagick (optional, informational only).

Companion files

- scripts/autonomous-qa/run-iteration.sh — produces the raw materials analyzed here
- scripts/autonomous-qa/record-qa-session.sh — the standalone recorder
- scripts/autonomous-qa/aggregate-evidence.sh — the JUnit-side rollup (this script is the vision-side complement)